

Лабораторная работа №4

Масштабирование чтения PostgreSQL: Primary + Replica

Цель работы

На практике настроить PostgreSQL Primary + Replica для собственного сервиса и понять идею Read Scaling: записи выполняются на Primary, а часть операций чтения может выполняться на Replica. В конце студент должен уметь объяснить, как изменения попадают на Replica и почему Replica может временно отставать от Primary.

Что должно получиться

- Primary — основной экземпляр БД, принимающий INSERT / UPDATE / DELETE.
- Replica — копия Primary, используемая для чтения.
- Между ними настроена streaming replication.
- Изменение, сделанное на Primary, появляется на Replica.
- Один реальный read-сценарий собственного backend-сервиса работает через Replica.

Часть 1. Поднять Primary и Replica

Измените окружение так, чтобы у сервиса появились два экземпляра PostgreSQL: Primary и Replica. Для лабораторной достаточно одного Primary и одной Replica.

Проверьте, что оба экземпляра запущены и доступны.

В отчёте покажите:

- фрагмент docker-compose.yml с двумя PostgreSQL;
- какой контейнер является Primary, а какой Replica;
- как подключиться к каждому экземпляру.

Часть 2. Настроить streaming replication

Настройте репликацию так, чтобы Replica получала изменения от Primary через WAL и воспроизводила их у себя.

Нужно понимать следующую цепочку:

- Primary изменяет данные.
- Изменение фиксируется в WAL.
- Replica получает поток WAL.
- Replica воспроизводит изменения.

Проверьте состояние репликации на Primary:

```
SELECT * FROM pg_stat_replication;
```

Часть 3. Доказать, что репликация работает

На Primary создайте или измените запись в выбранной таблице. Затем выполните SELECT на Replica и найдите эту запись.

В отчёте покажите:

- операцию записи на Primary;
- результат SELECT на Replica;
- скриншот или вывод, подтверждающий подключённую Replica.

Часть 4. Проверить read-only поведение Replica

Попробуйте выполнить INSERT или UPDATE на Replica. Зафиксируйте результат и объясните, почему Replica не должна использоваться приложением как независимая база для записи.

Часть 5. Направить чтение собственного сервиса на Replica

Добавьте в backend отдельное подключение к Replica.

Например:

- GET списка заказов;
- GET каталога;
- GET списка задач;
- GET профиля пользователя;
- любой другой SELECT из вашего сервиса.

Покажите в коде или конфигурации, что выбранный SELECT идёт на Replica. Операции записи вашего сервиса по-прежнему должны идти на Primary.

Часть 6. Понять replication lag

После INSERT/UPDATE на Primary сразу выполните SELECT через Replica. Если изменение уже применилось — это нормально. Если удастся увидеть момент, когда Primary уже содержит новое значение, а Replica ещё нет, зафиксируйте его.

Главный вывод: репликация не означает мгновенную синхронизацию. Между записью на Primary и применением изменения на Replica может существовать задержка — replication lag.

Контрольные вопросы

1. Чем Primary отличается от Replica?
2. Почему запись выполняем на Primary?
3. Как изменение из Primary попадает на Replica?
4. Что такое WAL в контексте репликации?
5. Что такое replication lag?

6. Почему следующий SELECT после INSERT потенциально может увидеть старые данные, если его отправить на Replica?
7. Что именно масштабируется при Read Scaling: скорость одного SQL-запроса или способность системы обслуживать больше чтений?
8. Почему наличие Replica не отменяет необходимость индексов и оптимизации SQL?
9. Расскажите про CAP-теорему